

Session 3 · Deck 3.B2

From Loops to Arrays

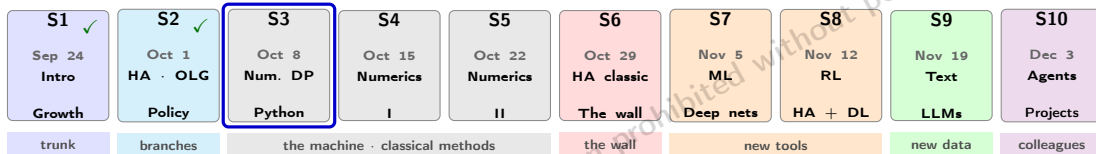
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.B2 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 3.B1 your toolkit · 3.B2 from loops to arrays · 3.B3 PyTorch, and coding with AI · 3.B4 the data workflow · 3.B5 hands-on: build it yourself.

Hands-on, 3.B5:

1 Version 0: grid VFI
against the closed form

2 Data → parameters:
 $g, n, \alpha, \delta, \beta$

3 The calibrated economy,
solved and checked

4 Test the model: untargeted
ratio, Euler residual

One Bellman Update, Written Three Ways

1. On the board

$$v_{n+1}(k_i) = \max_j \{U_{ij} + \beta v_n(k_j)\}$$

```
# 2. As you would say it out loud
for i in range(N):
    best = -1e18
    for j in range(N):
        val = U[i, j] + beta*V[j]
        if val > best:
            best = val
    Vn[i] = best

# 3. As NumPy wants to hear it
Vn = (U + beta*V[None, :]).max(axis=1)
```

Identical arithmetic. Identical answer —
 $\max|\text{difference}| = 0$, not “close”.

N	loop	array	speed-up
200	0.010 s	0.00004 s	263×
500	0.065 s	0.00018 s	373×
1000	0.261 s	0.00062 s	420×

Multiply the last row by the 272 sweeps of 3.A2:
1.2 minutes against 0.17 seconds, for the same number.

Best of repeated runs. The ratio grows with N :
NumPy's fixed cost per call is spread over more work.

Control Flow, and Where It Belongs

```
if r > rho:
    print("over-accumulation")
elif r < 0:
    print("negative real rate")
else:
    print("fine")

for n in range(max_iter):      # known count
    ...
while err > tol:               # unknown count
    ...

# a comprehension: shorter, still a loop
booms = [x for x in data if x > 0]
```

Indentation *is* the block structure — there are no braces, and a stray space is a syntax error.

The rule for this course

while belongs on the **outer** loop, where the iteration count is genuinely unknown: “keep sweeping until converged”.

The **inner** loops over states and choices should not be Python loops at all.

A comprehension reads as set-builder notation, $\{x : x \in \text{data}, x > 0\}$ — but it is **shorter, not faster**. Use it for structure, arrays for arithmetic.

Functions: Defaults, Docstrings, and Returning More Than One Thing

```
def cobb_douglas(k, z=1.0, alpha=0.36):  
    """Output per worker,  $y = z k^{\alpha}$ ."""  
    return z * k**alpha  
  
cobb_douglas(2.0)                # 1.283  
cobb_douglas(2.0, alpha=0.40)    # named  
  
def solve(model, tol=1e-6):  
    ...  
    return V, g, n                # a tuple  
  
V, g, n = solve(m)                # unpacked
```

- Default arguments put the **calibration** in one visible place.
- Named arguments at the call site make `solve(m, tol=1e-9)` readable at a glance.
- Returning a tuple is how one function hands back a value, a policy and a diagnostic.

Never use a **mutable** default (`def f(x, hist=[])`). It is created once and shared by every call — the reference-versus-copy trap of 3.B1 in its nastiest form.

Modules: When One File Stops Being Enough

```
import numpy as np
from scipy.optimize import brentq

# your own file, growth.py
from growth import GrowthModel

if __name__ == "__main__":
    # runs only when executed directly,
    # not when imported
    main()
```

A module is just a .py file. Splitting one gives you the project shape that survives to Session 5:

growth.py	the model and its methods
solvers.py	VFI, time iteration
run.py	calibration, output

The `__main__` guard is what lets `run.py` be both a script you submit to a scheduler and a module a notebook imports.

Why a Model Wants to Be a Class

A model is **state** plus **behaviour**. The state is α, β, δ , the grid, the chain; the behaviour is “take one Bellman step”, “solve”, “report the exact answer”.

Passing eleven loose variables into every function is how the wrong β ends up in the accuracy check. A class ties them together so that cannot happen.

Class and object. `GrowthModel` is the *class*: a blueprint. `m = GrowthModel(alpha, beta)` builds one *object* from it and runs its set-up once (next frame). Only then can you call `m.solve(tol)`.

Without

```
solve(k, U, beta, tol, alpha,  
z, Pi, ...)
```

With

```
m = GrowthModel(alpha, beta)  
m.solve(tol)
```

And it is the same pattern you will meet again in 3.B3: a PyTorch network *is* a class, holding parameters as state and a forward method as behaviour.

Anatomy of a Class: Setting Up the Model

```
class GrowthModel:
    """u(c)=ln c, y = z k^alpha, delta = 1."""

    def __init__(self, alpha=0.36, beta=0.95,
                  n=200, k_lo=0.05, k_hi=0.60):
        self.alpha, self.beta = alpha, beta
        self.k = np.linspace(k_lo, k_hi, n)

        C = (self.k**alpha)[: , None] \
            - self.k[None, :]
        self.U = np.full_like(C, -1e10)
        np.log(C, out=self.U, where=C > 0)
```

`__init__` runs once, when the object is made: `m = GrowthModel()` calls it for you, and you never call it yourself. Everything it assigns to `self` is available to every method afterwards.

Note *what* it precomputes: the payoff matrix U , which does not depend on v . Building it in the constructor is the same optimisation as 3.A2's "build it once, reuse it 272 times" — now enforced by the structure.

The Same Class, Now with Behaviour

```
def bellman(self, V):
    M = self.U + self.beta * V[None, :]
    return M.max(axis=1), M.argmax(axis=1)

def solve(self, tol=1e-6):
    V, n = np.zeros(self.k.size), 0
    while True:
        Vn, pol = self.bellman(V)
        d = np.max(np.abs(Vn - V))
        V, n = Vn, n + 1
        if d < tol:
            return V, self.k[pol], n

def exact_policy(self):
    return self.alpha*self.beta*self.k**self.alpha
```

```
m = GrowthModel()
V, g, n = m.solve()
n
# 272
np.max(np.abs(g/m.exact_policy() - 1))
# 0.009879
```

Same two numbers as this morning, to every digit. Restructuring code must not move a result — if it does, one of the two versions was wrong.

`exact_policy` lives *inside* the model, so the benchmark can never be evaluated at the wrong α .

NumPy: One Type, One Block of Memory

```
k = np.linspace(0.05, 0.6, 200)
k.shape, k.dtype      # (200,) float64
k.nbytes              # 1600

A = np.zeros((200, 200))
A.shape               # (200, 200)
A.nbytes              # 320000

s = k[10:20]          # a VIEW
s[0] = 99.0           # changes k!
c = k[10:20].copy()   # a copy
```

A Python list stores pointers to objects scattered in memory. A NumPy array stores **raw numbers**, **contiguously**, **all of one type** — which is what lets compiled code march through them.

A slice is a **view**, not a copy: 3.B1's reference-versus-copy rule, now with teeth. Writing into `V[10:20]` writes into `V`.

`dtype` is where 3.A1's `float64` decision actually gets made — and `np.float32` silently halves your precision.

Broadcasting: The Idea Behind the Whole Speed-Up

Two arrays are compatible if, reading shapes from the right, each pair of dimensions is **equal** or one of them is **1**. The size-1 dimension is then stretched — without ever being stored.

```
k[:, None].shape      # (200, 1)  rows
k[None, :].shape      # (1, 200)  columns

C = (k**alpha)[:, None] - k[None, :]
C.shape                # (200, 200)
# C[i, j] = k_i^alpha - k_j
```

(200, 1) $\Rightarrow$ stretched
(1, 200) $\Rightarrow$ stretched

(200, 200)

Every state against every choice, in one expression. That is the double loop of 3.A2, expressed as a shape.

The danger is that broadcasting almost never errors. Get an axis wrong and you get a plausible array of the wrong shape — so **assert the shape**.

The Vectorised Bellman Update, Line by Line

```
M = U + beta * V[None, :]  
#      (N,N) + (1,N) -> (N,N)  
#      M[i, j] = U[i, j] + beta*V[j]  
  
Vn = M.max(axis=1)      # over choices j  
pol = M.argmax(axis=1)  # which j won
```

With a shock the same idea carries one more axis:

```
EV = V @ Pi.T          # (N, nz)  
M = U + beta * EV.T[None] # (N, nz, N)  
Vn = M.max(axis=2)
```

The recipe

Give every index of the mathematics **its own axis**, let broadcasting form the full object, and reduce along the axis you were maximising over.

The expectation is a **matrix product**, $V @ \Pi.T$, done once per sweep — not re-summed inside the maximization, which is where the naive version wastes most of its time.

What Vectorising Does Not Buy You

The trick is to build the whole (N, N) object at once. So the memory bill is N^2 doubles, and it arrives all at once:

N	(N, N) float64	with a 7-state shock
200	0.3 MB	2.2 MB
1,000	8 MB	56 MB
10,000	800 MB	5.6 GB

The arithmetic is still $O(N^2)$ per sweep. Vectorising changed the *constant*, by a factor of a few hundred — it did not change the exponent.

The honest limit

A $400\times$ constant is two and a half orders of magnitude. The curse of dimensionality is many orders of magnitude. **Vectorising buys you one more state variable, not five.**

Which is why the next session attacks the exponent instead: exploiting monotonicity and concavity to avoid forming most of the matrix at all.

Measure Before You Optimise

```
%timeit m.bellman(V)
# prints the time per call

import cProfile
cProfile.run("m.solve()", sort="cumtime")
```

Three questions, in this order:

1. Is it **right**? Grade against the closed form first.
2. Is it **too slow to use**? If not, stop.
3. **Where** does the time go? Profile; never guess.

The two classic wastes

Recomputing invariants inside the loop — the payoff matrix, the transition matrix, anything built from parameters alone.

Optimising the wrong 5%. In this solver almost all the time is in one line; the rest is rounding error in your afternoon.

And record what you measured. A speed-up claim with no timing next to it is the same species of unsupported number as an accuracy claim with no benchmark.

Takeaways

- Python is slow only when **Python** does the arithmetic. The same Bellman update, written as one array expression, ran a few **hundred times faster** and returned a bit-for-bit identical answer.
- **Broadcasting** is the whole trick: give every index of the mathematics its own axis, let the shapes stretch, then reduce along the axis you were maximising over.
- It almost never errors. A wrong axis gives you a plausible array of the wrong shape, so **assert the shape** and check against a known answer.
- A model is state plus behaviour, so make it a **class** — and keep the closed-form benchmark inside it, where it cannot be evaluated at the wrong parameters.
- Vectorising changes the constant, not the exponent. A few hundred times is at most one extra state variable, and that is all.

Bridge: The Derivative You Never Have to Code

NumPy gave us the arithmetic. It does not give us derivatives — and from next week on, almost everything wants one: Newton's method needs a Jacobian, perturbation needs high-order derivatives at the steady state, and every neural network is trained by differentiating a loss.

The old answer was finite differences, with a step size you have to tune against exactly the round-off of 3.A1. The modern answer is to let the library track the arithmetic as you do it, and hand you the exact derivative afterwards.

Next (3.B3)

PyTorch from the ground up: tensors, automatic differentiation, `nn.Module`, and the training loop.

Then how to work with an AI assistant on all of this — and the one responsibility you cannot delegate.